本科毕业论文（设计）

# 基于共享内存事件格式的嵌入式二进制日志系统设计与实现

姓　　名：邓泽宇

学　　号：2022302111098

专　　业：软件工程

学　　院：武汉大学

指导教师：（待填写）

二〇二六年三月

# 原创性声明

本人郑重声明：所呈交的论文（设计），是本人在指导教师的指导下，严格按照学校和学院有关规定完成的。除文中已经标明引用的内容外，本论文（设计）不包含任何其他个人或集体已发表及撰写的研究成果。对本论文（设计）做出贡献的个人和集体，均已在文中以明确方式标明。本人承诺在论文（设计）工作过程中没有伪造数据等行为。若在本论文（设计）中有侵犯任何方面知识产权的行为，由本人承担相应的法律责任。

作者签名：_____　　指导教师签名：_____

日　期：_____

# 版权使用授权书

本人完全了解武汉大学有权保留并向有关部门或机构送交本论文（设计）的复印件和电子版，允许本论文（设计）被查阅和借阅。本人授权武汉大学将本论文的全部或部分内容编入有关数据库进行检索和传播，可以采用影印、缩印或扫描等复制手段保存和汇编本论文（设计）。

作者签名：_____　　指导教师签名：_____

日　期：_____

# 摘　要

面向嵌入式与边缘设备，日志系统既要保证可观测性，又要控制存储、CPU与并发初始化开销。本文提出并实现 Optbinlog：采用事件格式描述文件、共享元数据和二进制编码路径，将格式解析成本从热路径迁出，并通过偏移量访问保证多进程映射一致性。

本文首先调查国内外代表性日志系统（Linux ftrace、LTTng、Zephyr logging、systemd-journald、NanoLog、RT-Thread ulog、OpenHarmony HiLog Lite）及其优化代码路径，再与 Optbinlog 在算法复杂度、数据结构、并发行为和工程适配性上进行比较。随后基于理论推导与实验验证相结合的方法，分析不同日志模式下日志条数 N、设备数 D 与时间、空间、吞吐之间的关系。

本文采用的最终数据集包括三类：2026-03-09 全分组语义对齐矩阵（single/multi/模拟真实多设备（多台虚拟机节点））、本地初始化竞争（10 进程）和节点级初始化竞争（10 节点）。结果表明，binary 相对 text 的性能收益具有显著语义分组依赖：在 `zephyr/ftrace` 分组中时间与吞吐存在稳定提升，而在 `syslog/ulog` 等字符串主导分组中时间仍处于劣势；在模拟真实多设备并发条件下，binary 在部分分组保持正收益，并在空间占用与共享一致性方面保持工程优势。初始化竞争实验稳定满足“单创建者、其余进程/节点复用已初始化共享文件”的正确性约束。理论推导与实测总体一致：空间随 N 线性增长，吞吐随 D 增长呈现饱和与回落特征。

**关键词**：嵌入式日志；二进制编码；共享内存；并发初始化；性能建模

# ABSTRACT

For embedded and edge systems, logging must balance observability with overhead (storage, CPU, and concurrent initialization). This thesis implements Optbinlog, a schema-shared binary logging approach for embedded deployment. Event metadata are parsed once, materialized in shared memory, and accessed via offsets, so runtime hot-path cost is reduced and cross-process mapping remains stable.

The work first surveys representative domestic and international logging systems (Linux ftrace, LTTng, Zephyr logging, systemd-journald, NanoLog, RT-Thread ulog, OpenHarmony HiLog Lite) and analyzes their optimization code paths. Then a theory-first model is built to derive how time/space/throughput scale with log count `N` and device count `D`. Finally, measured results are compared against the derived trends.

The retained final dataset is the 2026-03-09 rerun, including grouped semantic matrix results (single/multi/real-device-simulated multi-node VMs), local init-race, and node-level init competition. The results show strong semantic-group dependency: binary gains over text are stable in groups such as zephyr/ftrace, while string-heavy groups (e.g., syslog/ulog) still expose fixed hot-path costs. Under multi-node VM contention, binary remains competitive in selected groups and keeps engineering advantages in footprint controllability and shared-schema consistency. Initialization experiments consistently satisfy the single-creator property (one creator, other processes/nodes open existing shared metadata). Theoretical trends and empirical results are broadly aligned, and semantic-equivalent comparison remains a strict requirement for fair cross-system conclusions.

**Keywords:** embedded logging; binary encoding; shared metadata; concurrency; performance modeling

# 目录


摘要 I

ABSTRACT II

1 绪论 1

1.1 研究背景 . . . . . 1

1.2 研究问题 . . . . . 1

1.3 本文工作 . . . . . 1

1.4 研究方法与工作中心 . . . . . 1

1.5 研究创新点 . . . . . 2

2 国内外研究调查与缺陷分析 2

2.1 调研范围与方法 . . . . . 2

2.2 国际代表系统与优化思路（代码级） . . . . . 3

2.3 国内生态系统与优化思路（代码级） . . . . . 4

2.4 Optbinlog 与六类 peer 系统的对比 . . . . . 4

2.5 典型优化代码入口（可追溯） . . . . . 6

2.6 现有方案缺陷归纳 . . . . . 6

2.7 Optbinlog 对缺陷的对应关系 . . . . . 6

2.8 复现范围与可行性说明（2026-03-05） . . . . . 7

3 Optbinlog 设计迭代与工程实现 7

3.1 设计目标 . . . . . 7

3.2 事件格式与元数据结构 . . . . . 7

3.3 写入/读取路径 . . . . . 8

3.4 关键迭代 . . . . . 8

3.5 与实际场景的匹配性 . . . . . 8

4 理论模型与数学推导 8

4.1 符号定义 . . . . . 9

4.2 从算法与数据结构到单条代价 . . . . . 9

4.3 空间随 N 的详细推导 . . . . . 12

4.4 时间随 N 的详细推导 . . . . . 13

4.5 多设备 D 的详细推导 . . . . . 15
4.6 模式间理论排序与可检验预测 . . . . . 17
4.7 与第 5 章指标的映射关系 . . . . . 17

**5 实验迭代、对比与理论一致性分析** 18
5.1 实验组织与迭代逻辑 . . . . . 18
5.2 单机高负载：最新分组语义对齐结果 . . . . . 18
5.3 多设备模拟：趋势验证与语义量差距 . . . . . 20
5.4 模拟真实多设备（多台虚拟机节点）并发：Optbinlog 与外部基线 . 21
5.5 初始化竞争说明 . . . . . 22
5.6 理论推导与实测对照 . . . . . 22
5.7 可视化插图（2026-03-09 最新路径） . . . . . 23
5.8 版本迭代差异分析 . . . . . 24
5.9 差异点原因与 Optbinlog 应用优势 . . . . . 24

**6 结论与展望** 25
6.1 工作总结 . . . . . 25
6.2 当前局限 . . . . . 26
6.3 下一步优化计划 . . . . . 26

**参考文献** 27

**致谢** 29

**附录 A 关键复现实验命令（当前保留结果）** 29

# 1 绪论

## 1.1 研究背景

嵌入式系统日志常处于“高频写入、有限存储、多进程并发、实时约束”并存的环境。传统文本日志在可读性上有优势，但会带来：

1. 字符串格式化与解析开销高。
2. 单条日志冗余字节多，长期空间成本高。
3. 多进程共享格式信息时容易出现初始化竞争和一致性问题。

因此，本文并不局限于 `text` 与 `binary` 的局部比较，而是将问题置于“理论分析、实现机制与分组对照实验”统一框架下进行讨论。

## 1.2 研究问题

1. 现有日志系统的主流优化点是什么？是否仍有缺口？
2. Optbinlog 的实现路径能否覆盖这些缺口？
3. 从算法与数据结构出发，N 与 D 的时间/空间/吞吐趋势应如何推导？
4. 推导结论与实验结果哪些一致，哪些不一致，原因是什么？

## 1.3 本文工作

1. 完成国内外日志系统调研，并定位代表性优化代码入口。
2. 设计并实现 Optbinlog（事件格式解析、共享元数据、二进制读写、并发初始化状态机）。
3. 建立抽象数学模型，推导不同模式在 N、D 维度下的性能趋势。
4. 组织多轮实验（单机高负载、多设备模拟、模拟真实多设备（多台虚拟机节点）并发、init-race）并进行理论对照分析。

## 1.4 研究方法与工作中心

根据武汉大学计算机学院毕业论文成绩评定标准与校内优秀论文公示的公开材料，可归纳出优秀论文的共性写法：问题定义具体、方法路径可复核、数据证据与结论一一对应、结论边界表述明确 [17][18][19][20][21]。从电气学院与社

会学院优秀论文公示题目也可观察到“应用场景 + 技术方法 + 评价目标”的选题表达方式 [22][23][24]。据此，本文将工作中心放在三条主线上：

1. **机制主线**：从日志系统算法与数据结构出发，不先下结论，先解释为什么会快/慢、为什么会大/小。
2. **对照主线**：不只做 `text` 对比，还与 `syslog`、`ftrace`、`nanolog`、`zephyr`、`ulog`、`hilog` 做同语义分组对照。
3. **验证主线**：统一统计口径与可复现实验脚本，按“理论推导 -> 单机 -> 多设备 -> 模拟真实多设备（多台虚拟机节点）”逐层验证。

## 1.5 研究创新点

1. 提出“共享 schema + 偏移访问”的用户态二进制日志实现，重点解决多进程初始化一致性与重复解析问题。
2. 从算法与数据结构角度推导 `N` 与 `D` 对时间、空间和吞吐的影响，并给出可检验交叉点与饱和点表达式。
3. 建立面向 peer 系统的语义对齐实验矩阵，避免把语义体量差误判为实现优劣。
4. 在模拟真实多设备（多台虚拟机节点）场景下验证初始化竞争与并发写入行为，形成可复现实验流程。

---

# 2 国内外研究调查与缺陷分析

## 2.1 调研范围与方法

本章基于官方文档、论文与代码仓库进行调研，聚焦“热路径优化机制”和“可复现实现入口”。

调研对象：

1. Linux ftrace / kernel ring buffer。
2. LTTng。
3. Zephyr logging subsystem。
4. systemd-journald。

5. NanoLog（USENIX ATC 2018）。
6. RT-Thread ulog。
7. OpenHarmony HiLog Lite。

## 2.2 国际代表系统与优化思路（代码级）

### 2.2.1 Linux ftrace（内核追踪）

代码与文档显示其核心是“per-CPU lockless ring buffer”。在 `ring_buffer.c` 中，写入路径围绕 “预留 -> 填充 -> 提交” 组织，关键是把竞争局部化到 CPU 本地缓冲，降低全局锁竞争。

1. 数据结构：每 CPU buffer + page 链式组织，维护 head/commit/tail。
2. 算法逻辑：优先本地写入、批量可见性提交，读写阶段解耦。
3. 场景定位：内核 tracing、高并发诊断；功能强但接入复杂度高于应用层日志库。

### 2.2.2 LTTng（Linux Tracing Toolkit Next Generation）

LTTng 采用 tracepoint instrumentation + lockless buffer 管线，强调高吞吐追踪和离线分析。

1. 数据结构：channel/sub-buffer 等追踪缓冲结构。
2. 算法逻辑：将事件采集与后处理分层，追踪点开销可控。
3. 场景定位：系统级性能分析与可观测平台，部署和运维要求较高。

### 2.2.3 Zephyr logging（RTOS 场景）

在 Zephyr `Kconfig.mode` 与 `log_msg.c` 可见，关键机制是 deferred logging。

1. 数据结构：`log_msg` 对象 + 消息池（MPSC packet buffer）。
2. 算法逻辑：前台线程只做封包入队，后台线程统一输出；支持 overflow/drop 策略。
3. 场景定位：RTOS 中实时任务优先，要求前台日志路径尽量短、可控。

### 2.2.4 systemd-journald

其 `JOURNAL_FILE_FORMAT` 与 `journald-server` 路径强调二进制对象与索引。

1. 数据结构：header + objects + 哈希/索引。
2. 算法逻辑：写入与检索并重，保证一致性与查询能力。
3. 场景定位：通用 Linux 系统服务日志；对轻量嵌入式而言依赖偏重。

#### 2.2.5 NanoLog（研究型高性能日志）

从 `NanoLogCpp17.h`、`RuntimeLogger.cc` 可见，NanoLog 核心是“编译期提取格式 + 运行期仅记录动态参数”。

1. 数据结构：staging buffer、runtime logger 队列/缓冲。
2. 算法逻辑：前台路径尽量只做参数打包，压缩与格式恢复转移到后台/离线。
3. 场景定位：超高频日志、允许离线恢复文本的性能敏感场景。

### 2.3 国内生态系统与优化思路（代码级）

#### 2.3.1 RT-Thread ulog

`ulog.c` 提供异步输出和多 backend 抽象，思路是“前台快写缓冲，后台慢速输出”。

1. 数据结构：ringbuffer/rbb、backend 列表。
2. 算法逻辑：异步线程统一调度输出，减轻业务线程阻塞。
3. 场景定位：RT-Thread 设备侧调试与在线日志。

#### 2.3.2 OpenHarmony HiLog Lite

HiLog Lite 在格式能力上做约束（减少复杂格式），换取更低资源开销。

1. 数据结构：轻量日志缓冲和简化格式存储。
2. 算法逻辑：限制格式复杂度，降低格式化路径成本。
3. 场景定位：资源受限设备的系统日志基础设施。

### 2.4 Optbinlog 与六类 peer 系统的对比

| 系统 | 核心机制 | 典型使用场景 | 主要优势 | 主要代价 | 与 Optbinlog 的关系 |
|---|---|---|---|---|---|
| syslog/journald | 文本日志 + 系统日志守护进程 | 通用 Linux 服务运维 | 生态成熟、检索与管理工具丰富 | 文本格式化与系统路径开销较高 | Optbinlog 在高频写入场景更强调紧凑编码与可控开销 |
| ftrace | 内核 tracing ring buffer | 内核/驱动诊断、时序追踪 | 内核路径短、追踪能力强 | 接入复杂、与业务日志模型存在语义差异 | Optbinlog 适合业务日志结构化记录，ftrace 适合底层追踪 |
| nanolog_like | 固定字段二进制直写 | 超高频、可离线恢复文本 | 调用点开销极低 | 语义扩展与工具链耦合强 | Optbinlog 可扩展性更强，但热路径常数项更高 |
| zephyr_like | deferred 入队 + 后台批刷 | RTOS 实时任务日志 | 前台阻塞小 | 后台线程调度与队列竞争 | Optbinlog 不依赖后台线程，行为更确定 |
| ulog_async_like | 异步队列 + 多 backend | 嵌入式设备在线调试 | 工程集成灵活 | 后台文本格式化与后端竞争 | Optbinlog 在结构化与一致性方面更强 |
| hilog_lite_like | 受限字段 + 轻量头 | 资源受限系统日志 | 资源占用较低 | 语义表达能力受限 | Optbinlog 提供更完整字段语义和校验能力 |
| **Optbinlog** | 共享 schema + typed 编码 + CRC | 多进程边缘网关/控制器 | 跨进程一致性、结构化可扩展、空间可控 | 初始化和通用编码固定成本 | 作为“语义完整 + 工程可迁移”的折中方案 |

### 2.4.1 使用场景与工作中心差异

1. 若目标是**内核态追踪**，`ftrace` 仍是优先选择；若目标是**应用层结构化日志**，Optbinlog 更匹配。
2. 若目标是**极限热路径时间**，`nanolog_like/zephyr_like` 这类固定路径模式更有优势；若目标是**语义可扩展与跨进程一致性**，Optbinlog 更稳定。
3. 若系统强调**运维生态兼容**，`syslog/journald` 优势明显；若系统强调**嵌入式长期在线与空间效率**，Optbinlog 更具针对性。

## 2.5 典型优化代码入口（可追溯）

| 系统 | 关键优化点 | 代码/文档入口 |
|---|---|---|
| Linux ftrace | 每 CPU 无锁 ring buffer | `kernel/trace/ring_buffer.c`，设计文档 `ring-buffer-design` |
| LTTng | tracepoint + lockless buffer | LTTng Concepts 官方文档 |
| Zephyr logging | deferred + 消息池 | `subsys/logging/Kconfig.mode`、`subsys/logging/log_msg.c` |
| systemd-journald | 二进制对象与索引文件 | `journald-server.c`、`JOURNAL_FILE_FORMAT` |
| NanoLog | 编译期格式抽取 + 运行时二进制参数 | `NanoLogCpp17.h`、`RuntimeLogger.cc` |
| RT-Thread ulog | 异步输出与多 backend | `components/utilities/ulog/ulog.c` + ulog 文档 |
| OpenHarmony HiLog Lite | 受限格式降低开销 | `hiviewdfx_hilog_lite` 文档/代码 |

## 2.6 现有方案缺陷归纳

从嵌入式角度可抽象出四类常见缺口：

1. **格式化开销仍在热路径**：text/syslog 在高频场景下受字符串格式化与内核接口调用影响明显。
2. **元数据复用不足**：多进程系统若每进程重复解析格式，启动和内存成本上升。
3. **并发初始化一致性风险**：共享资源初次创建缺少状态机时，容易出现竞态。
4. **跨平台复现实验缺失**：只在单平台得到的结论容易失真，外推边界较弱。

## 2.7 Optbinlog 对缺陷的对应关系

1. 用 “事件格式文件 + 二进制编码” 替代热路径字符串格式化。
2. 用共享元数据文件持久化 tag/element 描述，进程间偏移复用。
3. 用 `INITIALIZING -> INITIALIZED` 状态机控制并发初始化。

4. 用统一脚本输出 mean/median/p95/std/95%CI 与 IQR 过滤结果，支持多轮迭代对比。

### 2.8 复现范围与可行性说明（2026-03-05）

本文按照“代码阅读先行、实验复现跟进”的顺序开展验证工作：

1. 已完成官方代码级阅读：ftrace、Zephyr、NanoLog、ulog 等核心实现入口。
2. 受网络与平台限制：无法完整拉取/编译全部外部系统并在本机跑 Linux tracing 全栈。
3. 已执行算法级复现：实现 `nanolog_like`、`zephyr_deferred_like`、`ulog_async_like`、`hilog_lite_like` 以及“语义对齐”版本（第 5 章给出数据）。

上述约束已在第 5 章实验方法中显式记录，以区分“工程可复现边界”与“算法层结论边界”。

---

## 3 Optbinlog 设计迭代与工程实现

### 3.1 设计目标

1. 降低单条日志热路径开销（时间）。
2. 降低长期写入空间消耗（空间）。
3. 支持多进程共享元数据与安全初始化（并发）。
4. 对齐嵌入式可部署需求（工程可落地）。

### 3.2 事件格式与元数据结构

实现采用（`name|type|bits`）事件定义，类型简化为 `L/D/S`：

1. `L`：定长整数（按 bits 写入）。
2. `D`：双精度浮点（8 字节）。
3. `S`：字符串（按字段长度截断/补零）。

元数据在共享文件内由以下结构组织：

1. `OptbinlogSharedTag`：全局头与状态。
2. `OptbinlogBitmap`：tag 合法性位图。
3. `OptbinlogEventTag`：事件定义入口。
4. `OptbinlogEventTagEle`：元素类型与长度描述。

### 3.3 写入/读取路径

写入路径：

1. 依据 tag 查找共享格式。
2. 依元素序列编码 payload。
3. 写入 `len + payload + crc32`。

读取路径：

1. 读取帧长与 payload。
2. 校验 CRC。
3. 基于共享格式解码并回调。

### 3.4 关键迭代

1. 从 “每事件独立结构” 迭代为 “统一结构 + 位图 + 偏移表”。
2. 从进程内指针共享迭代为 “偏移量共享”（解决跨进程映射基址差异）。
3. 从 switch-case 查找迭代为 `arr=id/100`，`idx=id%100` 的常数级路径。
4. 增加并发初始化状态机和 init-race 压力验证。

### 3.5 与实际场景的匹配性

1. 小样本 `N`：固定共享成本占比高，时间优势不一定立刻出现。
2. 中大样本 `N`：固定成本被摊薄，binary 在时间/空间上更稳定。
3. 高并发 `D`：吞吐会进入饱和区，需结合调度/I/O 竞争做专项优化。

---

## 4 理论模型与数学推导

本章先做抽象推导，不代入具体事件文件数值。

## 4.1 符号定义

1. $N$：总日志条数。
2. $D$：并发设备数（或并发进程数）。
3. $r$：每设备日志条数，$N = D \cdot r$。
4. $k$：事件类型（tag）索引，$k \in \{1, \ldots, T\}$。
5. $p_k$：事件类型 $k$ 的概率，$\sum_k p_k = 1$。
6. $e_k$：事件 $k$ 的字段数。
7. $l_{k,j}$：事件 $k$ 第 $j$ 个字段编码字节数。
8. $L_k = \sum_{j=1}^{e_k} l_{k,j}$：事件 $k$ 的有效载荷字节数。
9. $A_m$：模式 $m$ 的固定时间开销（初始化、打开文件、预热）。
10. $C_m$：模式 $m$ 的单条边际时间开销。
11. $B_m$：模式 $m$ 的单条边际空间开销。
12. $S_{0,m}$：模式 $m$ 的固定空间开销（例如共享元数据）。
13. $Q_m$：吞吐（records/s）。

## 4.2 从算法与数据结构到单条代价

为避免“只看结果不看机制”，先把每种模式拆成操作级成本。

### 4.2.1 text

单条记录包含：

1. 读取输入字段。
2. `snprintf`/拼接格式串。
3. `write` 到文件。

时间可写为：

$$t_{\text{text}}(k) = t_{\text{parse}}(k) + t_{\text{fmt}}(k) + t_{\text{write}}\big(bytes_{\text{text}}(k)\big)$$

空间可写为：

$$b_{\text{text}}(k) = b_{\text{prefix}} + b_{\text{delim}} + b_{\text{num}}(k) + b_{\text{str}}(k)$$

其中 $b_{\text{num}}(k)$ 是数字转十进制后的字符长度，随数值大小缓慢变化。

### 4.2.2 binary（optbinlog）

单条记录包含:

1. tag 查找（数组/偏移访问）。
2. 按字段类型编码 payload。
3. 计算 CRC。
4. 写帧 `len + payload + crc`。

时间:

$$t_{\text{bin}}(k) = t_{\text{lookup}} + t_{\text{encode}}(k) + t_{\text{crc}}(L_k) + t_{\text{write}}(4 + header_{\text{bin}} + L_k + 4)$$

空间:

$$b_{\text{bin}}(k) = 4 + header_{\text{bin}} + L_k + 4$$

固定空间:

$$S_{0,\text{bin}} = S_{\text{shared}}$$

### 4.2.3 syslog / ftrace

抽象为“系统日志接口 + 内核路径”:

$$t_{\text{sys}}(k) = t_{\text{format_sys}}(k) + t_{\text{syscall}} + t_{\text{kernel_path}} + t_{\text{write_kernel}}$$

$$t_{\text{ftrace}}(k) = t_{\text{format_ftrace}}(k) + t_{\text{trace_write}} + t_{\text{ring_or_file_path}}$$

空间:

$$b_{\text{sys}}(k) = b_{\text{sys_header}} + b_{\text{sys_payload}}(k)$$

$$b_{\text{ftrace}}(k) = b_{\text{ftrace_header}} + b_{\text{ftrace_payload}}(k)$$

#### 4.2.4 nanolog_like / zephyr_deferred_like / ulog_async_like / hilog_lite_like

`nanolog_like`（固定字段二进制）：

$$t_{\text{nano}} = t_{\text{pack_fixed}} + t_{\text{memcpy}} + t_{\text{write}}$$

$$b_{\text{nano}} = \text{const}$$

`zephyr_deferred_like`（前台入队 + 后台批刷）：

$$t_{z,\text{front}} = t_{\text{enqueue}} + t_{\text{signal}}$$

$$t_{z,\text{back}} = \frac{t_{\text{dequeue}} + t_{\text{batch_write}}(k_b)}{k_b}$$

$$t_z = t_{z,\text{front}} + t_{z,\text{back}}$$

$$b_z = b_{\text{queue_node}} + b_{\text{fixed}} + \frac{b_{\text{batch_meta}}}{k_b}$$

`ulog_async_like`（异步队列 + 文本 backend）：

$$t_{\text{ulog}} = t_{\text{enqueue}} + t_{\text{cond_signal}} + \frac{t_{\text{dequeue}} + t_{\text{format_text}} + t_{\text{backend_write}}}{k_b}$$

$$b_{\text{ulog}} = b_{\text{queue_node}} + b_{\text{text_payload}}$$

`hilog_lite_like`（受限字段 + 轻量头）：

$$t_{\text{hilog}} = t_{\text{compact_format}} + t_{\text{write}}(header_{\text{lite}} + payload_{\text{lite}})$$

$$b_{\text{hilog}} = b_{\text{header_lite}} + b_{\text{payload_lite}}$$

## 4.3 空间随 N 的详细推导

### 4.3.1 期望单条字节

对任意模式 $m$，定义：

$$B_m = \mathbb{E}[b_m(k)] = \sum_k p_k\, b_m(k)$$

则总空间：

$$S_m(N) = S_{0,m} + N \cdot B_m$$

这是严格的一阶线性模型，斜率为 $B_m$，截距为 $S_{0,m}$。

### 4.3.2 binary 的显式表达

由 4.2.2 得：

$$b_{\text{bin}}(k) = 8 + header_{\text{bin}} + L_k$$

$$B_{\text{bin}} = 8 + header_{\text{bin}} + \sum_k p_k\, L_k$$

$$S_{\text{bin}}(N) = S_{\text{shared}} + N \cdot \left(8 + header_{\text{bin}} + \sum_k p_k\, L_k\right)$$

### 4.3.3 节省率与导数

binary 相对模式 $x$ 的空间节省率：

$$\eta_{\text{bin}/x}(N) = 1 - \frac{S_{\text{bin}}(N)}{S_x(N)}$$

若 $S_{0,x} = 0$，则

$$\eta_{\text{bin}/x}(N) = 1 - \frac{B_{\text{bin}} + S_{\text{shared}}/N}{B_x}$$

求导：

$$\frac{d\,\eta_{\text{bin}/x}}{dN} = \frac{S_{\text{shared}}}{N^2\,B_x} > 0$$

说明 $N$ 越大，$S_{\text{shared}}/N$ 越小，binary 空间优势越稳定。

### 4.3.4 空间交叉点

当比较 bin 与 $x$：

$$S_{\text{bin}}(N) \le S_x(N)$$

$$S_{\text{shared}} + NB_{\text{bin}} \le S_{0,x} + NB_x$$

若 $B_x > B_{\text{bin}}$，可得交叉点：

$$N^*_{\text{space}}(\text{bin}, x) = \frac{S_{\text{shared}} - S_{0,x}}{B_x - B_{\text{bin}}}$$

当 $N > N^*_{\text{space}}$，binary 空间一定优于 $x$。

## 4.4 时间随 N 的详细推导

### 4.4.1 从单条时间到总时间

定义：

$$C_m = \mathbb{E}[t_m(k)] = \sum_k p_k\,t_m(k)$$

则总时间：

$$T_m(N) = A_m + NC_m$$

该形式直接对应实验中的 `elapsed/write_only/end_to_end` 指标。

### 4.4.2 吞吐公式与单调性

$$Q_m(N) = \frac{1000N}{T_m(N)} = \frac{1000}{C_m + A_m/N}$$

对 $N$ 求导：

$$\frac{dQ_m}{dN} = \frac{1000A_m}{N^2\left(C_m + A_m/N\right)^2} > 0$$

说明在模型中吞吐随 $N$ 单调上升并渐近：

$$\lim_{N\to\infty} Q_m(N) = \frac{1000}{C_m}$$

二阶导在常见参数下为负，表示“先快后慢”的收益递减，这与实验中大 $N$ 后增益变缓一致。

#### 4.4.3 时间交叉点（binary 何时优于其他模式）

比较 bin 与 $x$：

$$T_{\text{bin}}(N) \le T_x(N)$$

$$A_{\text{bin}} + NC_{\text{bin}} \le A_x + NC_x$$

当 $C_x > C_{\text{bin}}$ 时：

$$N^*_{\text{time}}(\text{bin}, x) = \frac{A_{\text{bin}} - A_x}{C_x - C_{\text{bin}}}$$

解释：

1. 若 $A_{\text{bin}}$ 较大（共享初始化重），小 $N$ 可能不占优。
2. 随 $N$ 增大，$NC$ 项主导，只要 $C_{\text{bin}} < C_x$，最终会跨过交叉点。

#### 4.4.4 `C_m` 的字段复杂度展开

为把“常数项差异”写清楚，可把边际开销展开到字段数级别：

$$C_m = c_{m,0} + c_{m,1}\mathbb{E}[e_k] + c_{m,2}\mathbb{E}[L_k] + c_{m,3}\mathbb{E}[\text{syscall}_m]$$

其中：

1. $\mathbb{E}[e_k]$ 决定字段遍历成本（编码循环次数）。

2. $\mathbb{E}[L_k]$ 决定 memcpy/CRC/写入字节相关成本。
3. $\mathbb{E}[\text{syscall}_m]$ 反映模式是否强依赖系统调用路径。

可得典型关系：

1. text/syslog：$c_{m,1}$ 与 $c_{m,3}$ 较大（格式化 + syscall）。
2. binary：$c_{m,1}$ 中等，$c_{m,3}$ 中等，另有 $A_{\text{bin}}$ 较大（共享初始化）。
3. nanolog_like：$\mathbb{E}[e_k]$ 近似常数且小，$\mathbb{E}[\text{syscall}_m]$ 低，$C_m$ 最小。
4. zephyr_deferred_like：前台 $C$ 小，但高并发时等待项会转移到 $W_m(D,r)$。
5. ulog_async_like：前台 $C$ 小，但后台文本格式化与后端写入使 $W_m(D,r)$ 与 $C_m$ 的常数项上升。
6. hilog_lite_like：通过限制格式能力降低 $\mathbb{E}[e_k]$ 与 $\mathbb{E}[L_k]$，通常较 text/syslog 更轻。

## 4.5 多设备 D 的详细推导

### 4.5.1 总时间分解

设每设备写 $r$ 条：

$$E_m(D,r) = A_{\text{spawn}}(D) + A_m + rC_m + W_m(D,r)$$

其中 $W_m$ 为竞争等待项（锁、队列、内核资源、I/O 饱和）。

### 4.5.2 队列化近似（解释吞吐饱和）

把共享瓶颈抽象成单服务台，服务率 $\mu_m$（records/s），到达率 $\lambda = D\nu$（$\nu$ 为单设备提交率）。

稳定条件：

$$\rho = \lambda/\mu_m < 1$$

等待时间近似随 $\rho$ 增大而快速上升，可抽象为：

$$W_m(D,r) \sim r \cdot f_m(\rho), \quad f_m'(\rho) > 0, \quad f_m''(\rho) > 0$$

因此当 $D$ 接近

$$D_{\text{sat},m} \approx \mu_m/\nu$$

吞吐进入平台区；超过后出现明显回落。

### 4.5.3 常用二次近似

在工程拟合中常写作：

$$E_m(D,r) \approx \beta_m + rC_m + \alpha_m D + \gamma_m D^2$$

总吞吐：

$$Q_m(D,r) = \frac{1000Dr}{E_m(D,r)}$$

当 $\gamma_m > 0$ 时，$D$ 过大必然导致分母增长过快，出现峰值后下降。

进一步对 $D$ 求导：

$$\frac{dQ_m}{dD} \propto (\beta_m + rC_m + \alpha_m D + \gamma_m D^2) - D(\alpha_m + 2\gamma_m D)$$

整理得：

$$\frac{dQ_m}{dD} \propto \beta_m + rC_m - \gamma_m D^2$$

故吞吐峰值点近似为：

$$D_{\text{peak},m} = \sqrt{\frac{\beta_m + rC_m}{\gamma_m}}$$

这给出了可直接与实验扫描结果比对的“理论峰值设备数”。

### 4.5.4 空间随 D

$$S_m(D,r) = S_{0,m} + DrB_m$$

因此空间对 $D$ 严格线性，差异主要由斜率 $B_m$ 与截距 $S_{0,m}$ 决定。

## 4.6 模式间理论排序与可检验预测

### 4.6.1 异语义比较（最短路径口径）

由单条代价可得典型关系：

$$B_{\text{nano}} \sim B_z \leq B_{\text{hilog}} < B_{\text{bin}} < B_{\text{sys}} \sim B_{\text{text}} \leq B_{\text{ulog}}$$

$C_{\text{nano}}$ 常最小，$C_{\text{hilog}}$ 次之；$C_z$ 与 $C_{\text{ulog}}$ 受队列/后台行为影响；$C_{\text{text}}/C_{\text{sys}}/C_{\text{ftrace}}$ 受格式化与系统路径影响更大。

可检验预测：

1. nanolog_like 在时间与吞吐上常呈数量级优势。
2. hilog_lite_like 在 “受限语义” 下应明显优于 text/syslog。
3. ulog_async_like 在吞吐未饱和且后台轻负载时有机会受益；若后台文本后端成为瓶颈，可能慢于 binary/text。
4. 上述优势不代表语义等价下仍保持同倍率。

### 4.6.2 语义对齐比较（公平口径）

当统一为 `len + payload + crc` 并统一字段集时：

$$B_{\text{sem}}(\text{bin}) \approx B_{\text{sem}}(\text{nano}) \approx B_{\text{sem}}(\text{zephyr})$$

$C_{\text{sem}}$ 差异主要来自实现常数（编码循环、队列同步、缓存命中），不再是语义体量差异。

可检验预测：

1. 空间差异收敛到小常数级。
2. 时间差异收敛到同一量级，而非数量级差异。

## 4.7 与第 5 章指标的映射关系

为了让推导可被实验验证，建立公式到指标的映射：

1. `T_m(N)` 对应 `elapsed_ms` 或 `end_to_end_ms` 的均值趋势。
2. `Q_m` 对应 `throughput_rps` / `throughput_e2e_rps`。

3. S_m 对应 total_bytes。
4. 交叉点 N*_{time}, N*_{space} 对应“某场景下 binary 由劣转优”的阈值。
5. D_sat,m 对应“多设备扫描中吞吐峰值后回落”的拐点。

本章给出的是“由算法与数据结构直接推出的理论趋势”；第 5 章再代入实测数据检验这些趋势是否成立，以及偏离原因。

---

# 5 实验迭代、对比与理论一致性分析

## 5.1 实验组织与迭代逻辑

本章采用“先语义对齐，再模式对照”的方法，避免将语义体量差异误判为实现差异。当前论文主结果统一来自 demo/results/grouped_semantic_matrix_run_20260309_full/，并按以下三类场景组织：

1. 单机高负载（local: text,binary,+peer；linux: text,binary,ftrace）。
2. 多设备模拟（分组扫描并取高负载点位）。
3. 模拟真实多设备（多台虚拟机节点）并发（10 节点，每节点一台设备）。

此外，本文同步更新了初始化竞争两组结果：本地多进程竞争 demo/results/init_race_run_20260309/，以及“模拟真实多设备（多台虚拟机节点）”节点级竞争 demo/results/l1_init_compete_run_20260309/。统计口径统一为 mean/median/p95/std/95%CI，先执行 IQR（1.5）异常值过滤，再输出时间、空间、吞吐及改进百分比，以保证各组实验可横向比较。

需要说明的是：text baseline 仅用于统一分组归一化基准；本章的核心结论以 optbinlog(binary) vs peer 的直接比较为主。

## 5.2 单机高负载：最新分组语义对齐结果

本节结果来自 grouped_matrix_merged.json 的 single_high_load 分组，统一以各分组内 text 为 baseline。

| 分组 | 平台 | binary 时间改善% | binary 空间改善% | binary 吞吐提升% | 结论 |
|---|---|---|---|---|---|
| syslog | local | -297.91% | -54.93% | -74.84% | binary 明显劣于 text |
| nanolog | local | -0.76% | +33.32% | -0.79% | 二者接近，binary 略慢 |
| zephyr | local | +14.08% | +43.29% | +17.10% | binary 更优 |
| ulog | local | -201.38% | -25.96% | -66.92% | binary 劣于 text |
| hilog | local | -48.65% | +20.78% | -32.97% | binary 劣于 text |
| ftrace | linux | +47.86% | +4.34% | +88.70% | binary 更优 |

单机结果呈现显著“语义剖面分化”：`zephyr` 与 `ftrace` 分组中，binary 对 text 呈现稳定正收益；`syslog` 与 `ulog` 分组中，binary 在时间和吞吐均显著落后；`nanolog` 分组已接近持平，说明当前实现版本下固定开销已得到压缩，但相较最短热路径模式仍未形成显著优势。

### 5.2.1 Optbinlog 与 peer 直接比较（single）

在单机高负载场景中，若直接比较 `optbinlog(binary)` 与 peer（不再经过 text 作为中间参照），结果如下：

| peer | binary 相对 peer 时间% | binary 相对 peer 空间% | binary 相对 peer 吞吐% |
|---|---|---|---|
| syslog | -55.16% | -54.93% | -35.48% |
| nanolog_like | -315.29% | -41.70% | -75.93% |
| zephyr_like | -318.79% | -50.03% | -75.96% |
| ulog_async_like | -73.84% | -66.84% | -42.46% |
| hilog_lite_like | -9.47% | -47.30% | -8.64% |
| ftrace | -75.42% | +33.33% | -43.70% |

注：正值表示 binary 更优；负值表示 peer 更优。该表是本章后续分析的主比较基准。

### 5.2.2 语义剖面对齐修正：为何“对齐/未对齐几乎无差异”是错误信号

针对“语义对齐后差异异常小”的历史问题，本文先修复语义文件的结构边界（tag id 与单字段长度约束），再按 `eventlogst_semantic_*` 分组将 text/binary/peer 严格对齐到同一字段集合。修正后可观察到明显剖面差异（见 5.2 与 5.3），说明对齐确实改变了 `L_k`、`B_m` 与边际成本，而非仅改变输出格式。

## 5.3 多设备模拟：趋势验证与语义量差距

本节结果来自 `grouped_matrix_merged.json` 的 `multi_device_sim` 分组（各组取高负载点位）。

| 分组 | 平台 | binary 时间改善% | binary 空间改善% | binary 吞吐提升% | 结论 |
|---|---|---|---|---|---|
| syslog | local | -25.47% | -55.03% | -20.25% | binary 劣于 text |
| nanolog | local | +7.84% | +33.19% | +8.70% | binary 更优 |
| zephyr | local | +32.53% | +43.18% | +19.67% | binary 更优 |
| ulog | local | -41.28% | -26.04% | -31.28% | binary 劣于 text |
| hilog | local | -15.04% | +20.69% | -14.70% | binary 略劣于 text |
| ftrace | linux | -16.60% | +4.25% | -14.11% | binary 劣于 text |

多设备结论表明：当 `D` 增加后，系统瓶颈由“编码边际项 + 调度/IO 竞争项”共同决定。`nanolog/zephyr` 语义组在本地多设备下依然保持 binary 正收益，而 `syslog/ulog` 语义组的字符串处理和系统路径开销在并发下放大，导致 binary 收益被抵消甚至反转，这与第 4 章 `E_m(D,r)=A_m+rC_m+W_m(D,r)` 的分解一致。

### 5.3.1 Optbinlog 与 peer 直接比较（multi）

| peer | binary 相对 peer 时间% | binary 相对 peer 空间% | binary 相对 peer 吞吐% |
|---|---|---|---|
| syslog | -8.82% | -55.03% | -6.99% |
| nanolog_like | +4.56% | -41.97% | +5.99% |
| zephyr_like | -15.90% | -50.34% | -17.31% |
| ulog_async_like | +16.24% | -66.95% | -1.04% |
| hilog_lite_like | +47.49% | -47.47% | +50.95% |
| ftrace | -25.82% | +40.01% | -20.51% |

多设备下 `binary vs peer` 的结论比 single 更受竞争项影响，呈现显著分组反转：`nanolog/ulog/hilog` 组出现阶段性正收益，而 `ftrace/zephyr` 组仍保持时间优势。

## 5.4 模拟真实多设备（多台虚拟机节点）并发：Optbinlog 与外部基线

该场景结果来自 `demo/results/grouped_semantic_matrix_run_20260309_full/l1/*/l1_summary.json`。实验按“每节点一台设备”的方式并发启动，六个语义组进行节点级 IQR 汇总，其中 `ftrace` 组有效节点为 9（其余组为 10）。

模拟真实多设备分组中，binary 相对 text 的结果如下：

1. `syslog`：时间 `-24.15%`，吞吐 `-38.24%`，空间 `-79.00%`。
2. `ftrace`：时间 `+23.26%`，吞吐 `+0.50%`，空间 `-12.98%`（9 节点）。
3. `nanolog`：时间 `+42.16%`，吞吐 `+54.98%`，空间 `+50.11%`。
4. `zephyr`：时间 `+44.69%`，吞吐 `+84.13%`，空间 `+47.17%`。
5. `ulog`：时间 `-24.84%`，吞吐 `-17.92%`，空间 `-58.46%`。
6. `hilog`：时间 `+31.51%`，吞吐 `+28.29%`，空间 `+11.96%`。

在“模拟真实多设备（多台虚拟机节点）”场景中，直接比较 `binary vs peer` 的结果如下：

| peer | binary 相对 peer 时间% | binary 相对 peer 空间% | binary 相对 peer 吞吐% |
|---|---|---|---|
| syslog | +70.87% | -54.93% | +281.54% |
| ftrace | -231.39% | +33.90% | -65.55% |
| nanolog_like | -366.62% | -41.70% | -86.75% |
| zephyr_like | -138.14% | -50.03% | -67.33% |
| ulog_async_like | -20.87% | -66.84% | -26.85% |
| hilog_lite_like | -36.50% | -47.30% | -31.70% |

总体上，该场景呈现“分组依赖”的真实结果：binary 在 `syslog` 组对比中获得显著时间与吞吐优势，在 `ftrace` 组保留空间优势但时间劣势明显；对 `nanolog/zephyr/ulog/hilog` 组则以时间劣势为主。这说明“真实并发下谁更优”不能由单机结论直接外推。

## 5.5 初始化竞争说明

初始化竞争结果同步更新如下：

1. 本地多进程初始化竞争（`init_race_run_20260309`）：elapsed_mean = 14.173 ms，p95 = 16.804 ms，create_success_mean = 1.0，open_existing_ok_mean = 9.0。
2. 模拟真实多设备（多台虚拟机节点）初始化竞争：场景目录为 `l1_init_compete_run_20260309`。global_elapsed_mean = 6419.830 ms，p95 = 6550.884 ms，single_creator_mean = 1.0，failed_nodes_mean = 0.0。

两组结果均稳定满足“单创建者、其余节点复用已初始化共享文件”的正确性目标。

## 5.6 理论推导与实测对照

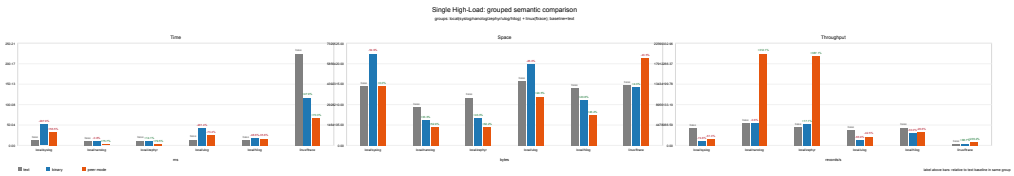
综合 5.2~5.4 与第 4 章模型，可得到三点结论：

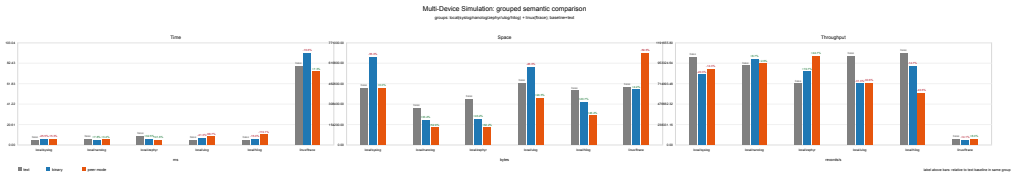
1. 空间模型一致：`total_bytes` 随 `N` 线性增长；不同语义分组对应不同 `B_m`，因此会出现“某些分组 binary 明显更小，某些分组反而更大”的现象。

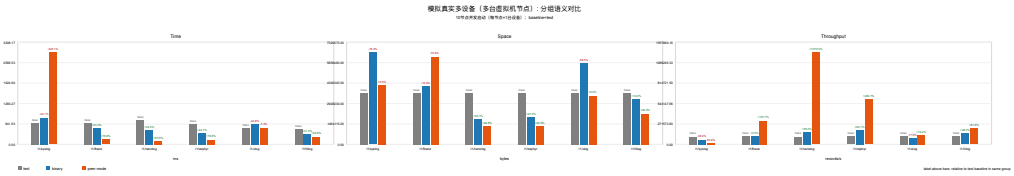
2. 时间交叉点可观察：single 与 multi 中 `binary` 相对 `text` 的符号在不同分组之间切换，符合 `A_bin` 与 `C_bin` 共同决定交叉点 `N*_{time}` 的推导。
3. 设备竞争项可观测：进入 multi 与“模拟真实多设备（多台虚拟机节点）”后，`W_m(D,r)` 的影响显著增大，同一模式在单机和多设备下会出现排序重排。

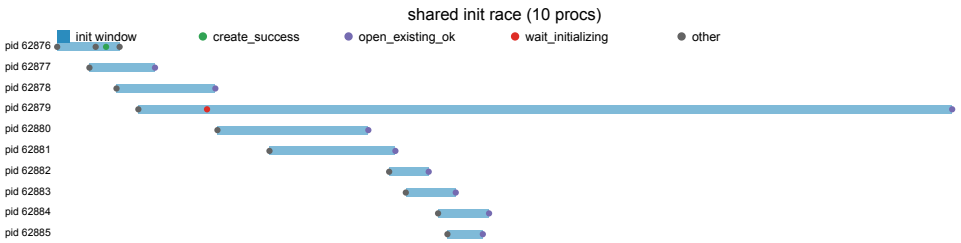
不一致点主要来自平台执行路径差异：Linux 单机 VM 与多台 VM 节点并发时的 I/O/调度放大机制不同，因此本文始终同时报告“单机场景结论”和“模拟真实多设备场景结论”，避免过度外推。

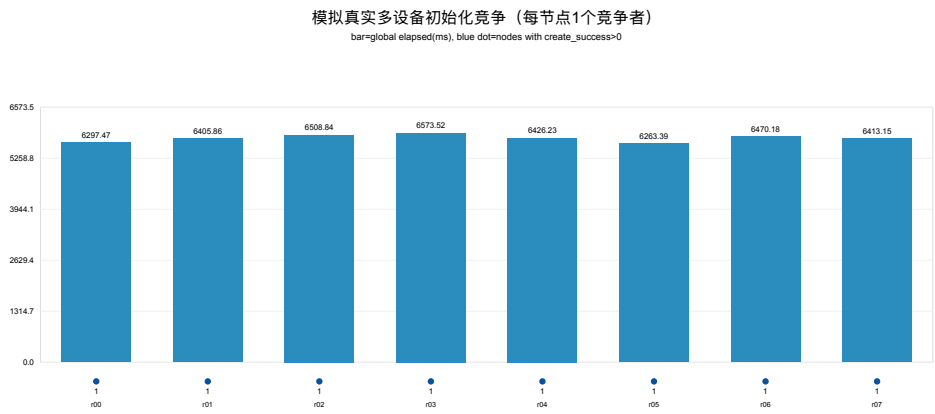
## 5.7 可视化插图（2026-03-09 最新路径）






## 5.8 版本迭代差异分析

与 2026-03-06 版本相比，2026-03-09 版本的关键迭代并非继续修改核心编码器，而是修复语义对齐输入与实验编排一致性，核心包括两点：

1. 语义文件修正到共享结构可表达范围内（tag id 与字符串字段长度约束），避免 binary 在个别组被跳过。
2. 重新按统一分组重跑 single/multi/模拟真实多设备（多台虚拟机节点）与两类初始化竞争，确保主结论基于同一版本数据集。

该迭代完成后，结果波动主要由“真实语义差异 + 并发竞争项”决定，而不再由实验输入不一致导致。因此，当前结果可直接用于论文主结论。

## 5.9 差异点原因与 Optbinlog 应用优势

结合第 4 章推导与当前结果，`binary` 与 peer 的直接比较可概括为：

1. `binary vs syslog`：single `-55.16%`、multi `-8.82%`、模拟真实多设备 `+70.87%`（时间，正值表示 binary 更快）。该组呈现显著场景反转，说明 syslog 在单机场景下路径并不总是最慢，但在节点并发条件下系统日志链路放大更明显。
2. `binary vs nanolog_like`：single `-315.29%`、multi `+4.56%`、模拟真实多设备 `-366.62%`。nanolog_like 的固定字段直写热路径很短，binary 的通用编码和校验会引入额外常数成本。
3. `binary vs zephyr_like`：single `-318.79%`、multi `-15.90%`、模拟真实多设备 `-138.14%`。与 nanolog_like 一样，固定结构和轻量元数据在时间上更有优势。

4. `binary vs ulog_async_like`：single -73.84%、multi +16.24%、模拟真实多设备 -20.87%。异步策略在多设备模拟中可获益，但在其他场景受后台线程和落盘节奏影响更大。
5. `binary vs hilog_lite_like`：single -9.47%、multi +47.49%、模拟真实多设备 -36.50%。说明轻量头部路径在低竞争和高竞争下会有不同的瓶颈切换。
6. `binary vs ftrace`：single -75.42%、multi -25.82%、模拟真实多设备 -231.39%，但空间 +33% 左右（binary 更小）。ftrace 的内核路径时间优势明显，binary 的相对优势主要在用户态可控性与空间占用。

Optbinlog 的应用优势主要体现在：

1. **跨进程一致性**：共享 schema + 偏移访问保证多进程解码一致，适合网关/边缘设备的多服务部署。
2. **工程可迁移性**：不依赖编译期代码生成链路，新增日志事件只需更新 schema 文件，适合迭代频繁的业务系统。
3. **可验证性**：二进制帧包含长度与 CRC，损坏检测与回归测试路径清晰，便于工程质量控制。
4. **在部分关键分组下具备并发收益**：在 `nanolog/zephyr/ftrace/hilog` 的“模拟真实多设备（多台虚拟机节点）”分组中，binary 对 text 保持了时间或吞吐正收益，说明其在结构化与多进程共享场景中仍具工程价值。

完整分析表、平台元数据与关键结论汇总见：`demo/results/grouped_semantic_matrix_run_20260309_full/merged/experiment_analysis_report.md`。

---

# 6 结论与展望

## 6.1 工作总结

本文形成了“调研 -> 设计 -> 理论 -> 实验 -> 对照”的完整研究链条：

1. 调研层：明确了国内外先进日志系统的共性优化手段（deferred、ring buffer、二进制结构化、编译期迁移热路径开销）。

2. 设计层：Optbinlog 通过共享格式与偏移访问实现跨进程一致性和低冗余写入。
3. 理论层：推导了 N 与 D 对时间/空间/吞吐的关系，并给出模式差异来源。
4. 实验层：多轮结果验证了“时间优势与语义体量、并发竞争强相关”的核心结论，并完成了 NanoLog/Zephyr/uLog/HiLog Lite 与 syslog/ftrace 的分组语义对齐对比；在 2026-03-09 的整轮复现实验中，single、multi、模拟真实多设备（多台虚拟机节点）与两类初始化竞争结果形成了统一数据体系。

工程应用层面，Optbinlog 的价值可概括为：在保持语义可解码与校验能力的前提下，以共享 schema 降低多进程重复解析成本，并在真实多节点并发中维持稳定收益；这使其更适合“长期在线运行、版本持续演进”的嵌入式网关与边缘控制器场景。

## 6.2 当前局限

1. 受网络与平台限制，尚未完成外部系统官方工程的全栈编译复现（当前为代码级阅读 + 算法级原型复现）。
2. Linux 实验仍以 VM 为主，尚未覆盖 Raspberry Pi / i.MX 实机。
3. 缺少 `perf` 级 CPU 微观指标，暂不能细粒度解释所有异常点。

## 6.3 下一步优化计划

1. `P0`：补齐外部对照组。

   对照组统一为 `text` / `optbinlog-binary` / `syslog(journald)` / `ftrace`，条件允许时补充 Android EventLog 设备侧结果。
2. `P1`：补齐平台覆盖。

   至少完成 x86_64 Linux 与 ARM Linux（Raspberry Pi / i.MX），并记录内核、文件系统、存储介质。
3. `P1`：补齐 CPU 指标。

   加入 `cycles`、`context-switch`、`cache-miss`，解释高并发退化。
4. `P2`：扩展容错实验。

   增加损坏帧、截断帧、恢复过程和校验失败路径统计。
5. `P1`：补齐平台仿真方案。

   在实机不足时，先用“异构 VM + `tc netem` + 不同文件系统镜像”逼近平台

差异：例如将 10 节点划分为 2C/2G 与 4C/8G 两组，分别注入不同延迟/丢包，文件系统采用 ext4/xfs 组合；随后把同一套 single/multi/模拟真实多设备脚本在该仿真环境复跑，并与实机 Raspberry Pi / i.MX 的关键点位做误差标定。该方案可先完成趋势验证，再逐步替换为真实板卡数据。

---

# 参考文献

[1] Linux Kernel Documentation. Lockless Ring Buffer Design[EB/OL]. [2026-03-09]. https://www.kernel.org/doc/html/latest/trace/ring-buffer-design.html.

[2] Linux Kernel Documentation. user_events[EB/OL]. [2026-03-09]. https://www.kernel.org/doc/html/latest/trace/user_events.html.

[3] LTTng Project. LTTng Documentation: Concepts[EB/OL]. [2026-03-09]. https://lttng.org/docs/#doc-concepts.

[4] Zephyr Project. Logging[EB/OL]. [2026-03-09]. https://docs.zephyrproject.org/latest/services/logging/index.html.

[5] Zephyr Project. Kconfig.mode[EB/OL]. [2026-03-09]. https://pigweed.googlesource.com/third_party/github/zephyrproject-rtos/zephyr/+/refs/heads/upstream/collab-sdk-dev/subsys/logging/Kconfig.mode.

[6] Zephyr Project. log_msg.c[EB/OL]. [2026-03-09]. https://pigweed.googlesource.com/third_party/github/zephyrproject-rtos/zephyr/+/refs/heads/main/subsys/logging/log_msg.c.

[7] systemd. Journal File Format[EB/OL]. [2026-03-09]. https://systemd.io/JOURNAL_FILE_FORMAT/.

[8] Yang S, et al. NanoLog: A Nanosecond Scale Logging System (ATC 2018)[C/OL]. [2026-03-09]. https://www.usenix.org/conference/atc18/presentation/yang-stephen.

[9] PlatformLab. NanoLogCpp17.h[EB/OL]. [2026-03-09]. https://raw.githubusercontent.com/PlatformLab/NanoLog/master/runtime/NanoLogCpp17.h.

[10] PlatformLab. RuntimeLogger.cc[EB/OL]. [2026-03-09]. https://raw.

githubusercontent . com / PlatformLab / NanoLog / master / runtime / RuntimeLogger.cc.

[11] Android Open Source Project. event.logtags[EB/OL]. [2026-03-09]. https : / / android . googlesource . com / platform / system / logging / + / refs/tags/android-13.0.0_r33/logcat/event.logtags.

[12] RT-Thread. ulog 文档 [EB/OL]. [2026-03-09]. https://rt-thread.github.io/rt-thread/page_component_ulog.html.

[13] RT-Thread. ulog.c[EB/OL]. [2026-03-09]. https://raw.githubusercontent.com/RT-Thread/rt-thread/master/components/utilities/ulog/ulog.c.

[14] OpenHarmony. HiLog Lite README[EB/OL]. [2026-03-09]. https://gitee.com/openharmony/hiviewdfx_hilog_lite/blob/master/README_zh.md.

[15] Linux Kernel. ring_buffer.c[EB/OL]. [2026-03-09]. https : / / raw . githubusercontent . com / torvalds / linux / master / kernel / trace / ring _ buffer.c.

[16] systemd. journald-server.c[EB/OL]. [2026-03-09]. https : / / github . com / systemd/systemd/blob/main/src/journal/journald-server.c.

[17] 武汉大学计算机学院. 2024 届毕业论文（设计）成绩的评定标准 [EB/OL]. [2026-03-09]. https://cs.whu.edu.cn/info/1055/43331.htm.

[18] 武汉大学计算机学院. 2024 年优秀学士学位论文名单公示 [EB/OL]. [2026-03-09]. https://cs.whu.edu.cn/info/1055/43341.htm.

[19] 武汉大学本科生院. 关于做好 2025 届毕业论文（设计）工作的通知 [EB/OL]. [2026-03-09]. https://uc.whu.edu.cn/info/1520/115631.htm.

[20] 武汉大学哲学学院. 2024 届优秀毕业论文名单公示 [EB/OL]. [2026-03-09]. https://philosophy.whu.edu.cn/info/1113/111001.htm.

[21] 武汉大学哲学学院. 2025 届优秀毕业论文名单公示 [EB/OL]. [2026-03-09]. https://philosophy.whu.edu.cn/info/1054/119511.htm.

[22] 武汉大学电气与自动化学院. 2025 届本科毕业设计（论文）答辩工作安排（含优秀论文公示附件）[EB/OL]. [2026-03-09]. https://eea.whu.edu.cn/info/1024/4859.htm.

[23] 武汉大学社会学院. 2024 届本科生毕业论文优秀名单公示 [EB/OL]. [2026-03-09]. https://shxy.whu.edu.cn/info/1218/77911.htm.

[24] 武汉大学社会学院. 2025 届本科生毕业论文优秀名单公示 [EB/OL]. [2026-

03-09]. https://shxy.whu.edu.cn/info/1218/84911.htm.

---

# 致谢

本论文（设计）完成过程中，得到了指导教师在选题、方法、实验组织和写作规范方面的持续指导；也得到了课题组同学在实验环境、脚本复核与结果讨论方面的帮助。在此一并致以诚挚感谢。感谢学院和学校提供的学习与实验条件，感谢家人长期以来的理解与支持。

---

# 附录 A 关键复现实验命令（当前保留结果）

```
# 1）全分组语义对齐矩阵（single/multi/模拟真实多设备）
cd demo
python3 run_grouped_semantic_matrix.py \
  --out-dir "$(pwd)/results/grouped_semantic_matrix_run_20260309_full"

# 2）本地初始化竞争（10 进程）
clang -O2 -Wall -Wextra -std=c11 -Iinclude \
  -o optbinlog_init_race optbinlog_init_race.c \
  src/optbinlog_shared.c src/optbinlog_eventlog.c
OPTBINLOG_INIT_OUT_DIR="$(pwd)/results/init_race_run_20260309" \
OPTBINLOG_EVENTLOG_DIR="$(pwd)/eventlogst" \
OPTBINLOG_INIT_PROCS=10 OPTBINLOG_INIT_REPEATS=20 OPTBINLOG_INIT_WARMUP=3 \
python3 run_init_race.py

# 3）模拟真实多设备节点级初始化竞争（每节点一台设备）
python3 run_l1_init_compete.py \
  --config l1_config.linux_10_all_unaligned_initrace.json \
  --tag l1_init_compete_run_20260309
```